

Lecture 2 • Wed 9/30/2026

CS160 Compilers — Fall 2026

Your First Compiler.

Yu Feng

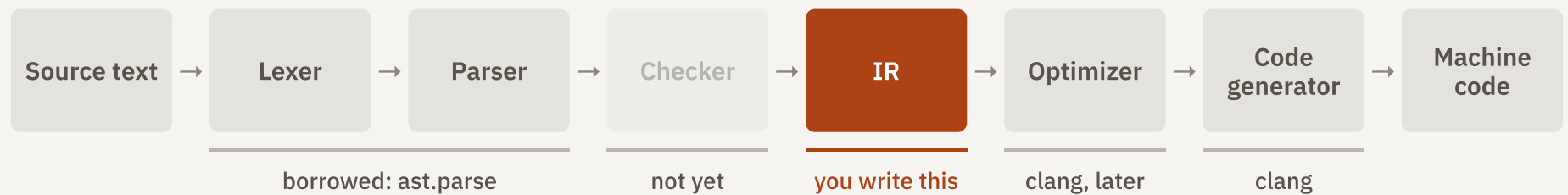
How does a program become something a CPU can run?

By the end of the hour you will have seen every step, and written the one that matters.

You can read a program as a *tree*, and write that tree out as a list of *instructions* — single steps the machine can carry out.

And you'll have a real **binary** — a file your laptop can start and run on its own — compiled from a language we finish designing today.

One box is yours today.



Lexer, Parser — Python's `ast.parse`, borrowed until week 3.

Checker — no *type* can go wrong yet: everything is an int.

Optimizer — clang, later. **Code generator** — clang.

It's the one that matters.

ChocoPy, day one.

Integer literals. `+` `-` `*` `//` `%`. Parentheses.
Negation. `print(...)`. One statement
per line.

```
print(3 * (3 + 4))  
print(10 // 3 - -2)
```

No variables until Monday. No types to check because everything is an int.

Today's entire language is two sections of the manual — **§3.4.2** (integer literals) and **§3.5** (operators). It grows one section at a time.

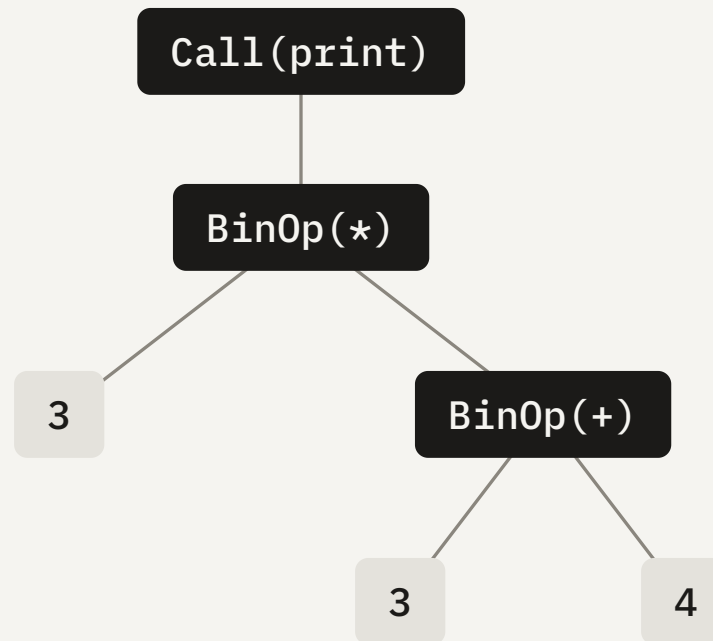
`ast.parse` gives us the tree for free.

```
>>> import ast
>>> print(ast.dump(ast.parse("print(3 * (3 + 4))"), indent=2))
Expr(
  value=Call(
    func=Name(id='print'),
    args=[
      BinOp(
        left=Constant(value=3),
        op=Mult(),
        right=BinOp(
          left=Constant(value=3),
          op=Add(),
          right=Constant(value=4))))])
```

This is your input. Not text — a tree. The parentheses are already gone.

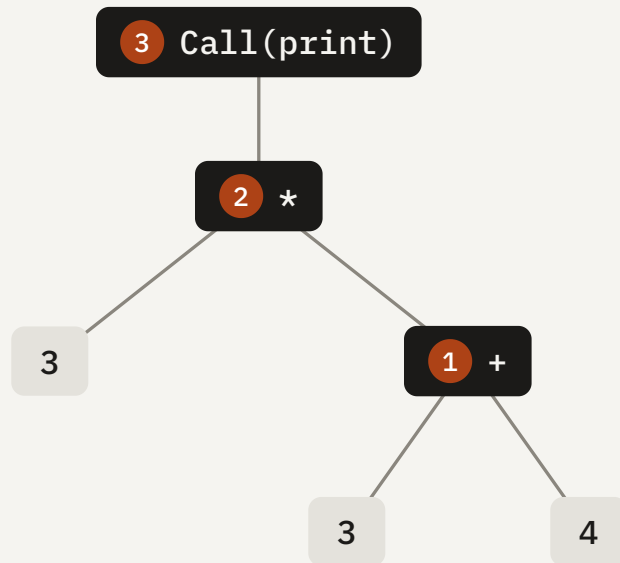
But it is *Python's* parser, and Python is not ChocoPy. It accepts programs the manual doesn't: `3 < 4 < 5` parses here and is not a ChocoPy program at all (§4.1 — comparisons don't chain). Until week 3 we simply don't write those.

The tree, drawn.



Leaves are numbers. Inner nodes are operators. Evaluate bottom-up.

Evaluate bottom-up, write each step down.

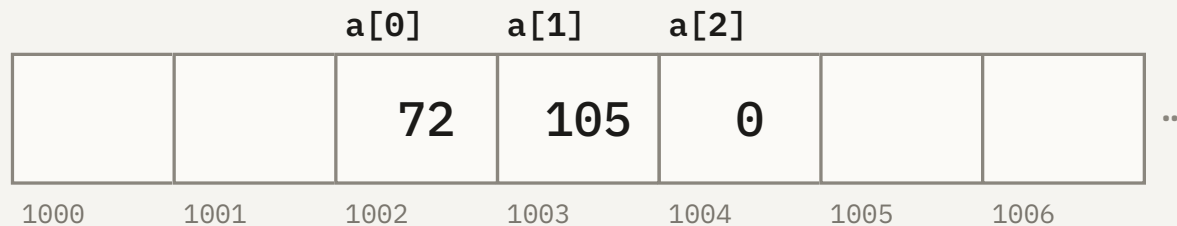


```
1  t1 = 3 + 4
2  t2 = 3 * t1
3  print(t2)
```

A tree, flattened. Each line is one **instruction** — one operation, naming its result. This is the *middle form* Lecture 1 called an **IR**, and slide 9 gives it its full name.

Three facts about the machine. Today needs no others.

memory · numbered bytes



what the machine is holding right now



1 • Memory is one enormous array of numbered bytes. A byte's number is its **address** — which is all a C pointer ever was.

2 • One instruction at a time, in order. What changes that order is an instruction saying *continue from this named position instead*: the name is a **label**, the instruction is a **jump**. Calling a function is the same move with a note left behind saying where to come back to — week 5. There is nothing else.

3 • A few values at a time. Outside memory the machine can hold only a few values; we will call those *the values the machine is holding*. Slide 7's t1 and t2 are them. Week 10 gives them their real name.

You had the first one before you walked in. The other two are why the next slide looks the way it does — and the second one is **Monday's entire second half**.

C with one operation per line, and no **if**.

Slide 7's list is what we want to produce — but nothing runs it. `t1 = 3 + 4` is not a program in any language. We need a form some real tool will finish for us.

LLVM is a toolkit for building compilers, and its **IR** — *intermediate representation*, the middle form between the tree the front end makes and the machine code the back end makes — is that form. The quickest honest description: **it is C with one operation per line, every value labelled with its type, and no `if` or `while` — only `goto`.**

clang is the program that reads it and produces something your laptop can actually run.

Writing x86 or ARM by hand is weeks of work that teaches you almost nothing about compiling, so we stop here and let clang finish. Rust, Swift and Julia stop in exactly the same place. **This is the design, not a shortcut.**

The manual is the spec.

The language is **ChocoPy**, and its published reference manual is the spec — PA1 will point you at it.

This quarter implements a growing subset of it: no classes, no `for`; strings and lists arrive with PA4.

Where what you build departs from the manual — and page 1 of your PA1 handout makes it depart on two points — the slide that does it will say so.

LLVM IR: the same list, in LLVM's spelling.

hello.ll · compiles and prints 21

```
declare void @print_int(i32)
```

← **declare** a function that lives somewhere else — no body, just a promise about its shape. `@print_int` is in our tiny C runtime (slide 13).

```
define i32 @main() {
```

← **define** here is a function and its body; `main` is where the program starts. `i32` 32-bit integer — every value carries its **type**.

```
entry:
```

← a **label**, exactly as on slide 8 — and this is where execution starts.

```
    %t1 = add i32 3, 4
```

← operation, type, two operands.

```
    %t2 = mul i32 3, %t1
```

← `%t1` a **temporary** — a name for one result; slide 8's values the machine is holding.

```
    call void @print_int(i32 %t2)
```

```
    ret i32 0
```

← leave this function, handing back 0. Nothing today depends on the number.

```
}
```

Five things to carry out of here.

- 1 Every value has a **type**, so clang can pick the right instruction and catch your mistakes. Today every value is an `i32` — a 32-bit integer. Booleans arrive Monday; a real type checker in week 5.
- 2 `%name` is a temporary; `@name` is a function.
- 3 One instruction, one operation, one result.
- 4 Each `%name` is written **once**. Need another result, make another name — which is why the code on slide 15 hands them out from a counter. Why it has to be that way: week 8.
- 5 It is a text file. You write it with `print()`.

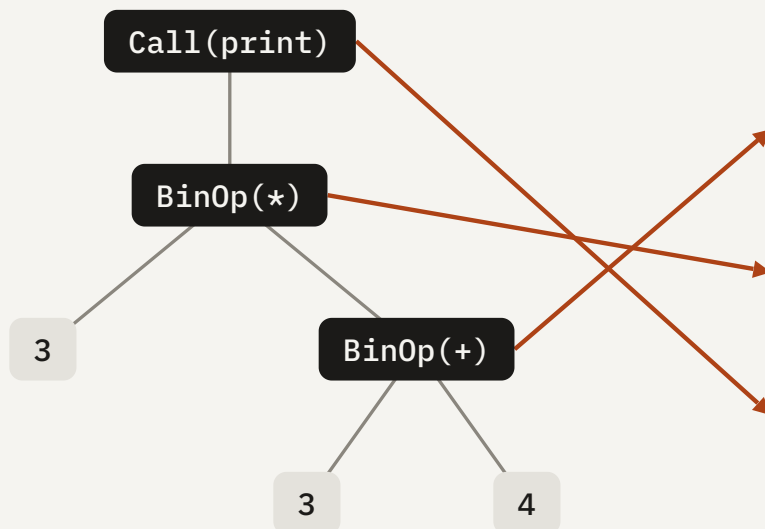
Where does `print_int` come from?

```
// runtime.c — provided by the staff
#include <stdio.h>
void print_int(int x) { printf("%d\n", x); }
```

`clang hello.ll runtime.c -o hello`. Your IR and our C become one binary.

Two files, one program: clang translates each, then **links** them — glues the results together and matches your `@print_int` up with ours. Spell that name wrong and you get the first error most of you will meet in PA1.

Turn the left into the right.



```
%t1 = add i32 3, 4
%t2 = mul i32 3, %t1
call void @print_int(i32 %t2)
```

Each **operator** node becomes one instruction.

Each **number** becomes *no* instruction — it is written straight into the instruction that uses it.

So the compiler is one function: given a node, write out its instructions and **return the name of its result** — "3" for a number, "%t1" for an operator.

Four pieces, 68 lines. We follow this one example through every piece.

The Emitter.

```
class Emitter:
    def __init__(self):
        self.lines = []      # instructions, in order
        self.n = 0          # counter for fresh temporaries
```

after compiling `print(3 * (3 + 4))`

```
n      = 2
lines = [ "%t1 = add i32 3, 4",
          "%t2 = mul i32 3, %t1",
          "call void @print_int(i32 %t2)" ]
```

Two jobs: keep the instructions in order, and invent names nobody has used yet — `fresh()` returns `%t1`, then `%t2`, then `%t3`.

The leaf case: no instruction at all.

```
def compile_expr(e, out):  
    """Write e's instructions into out; return the name of e's value."""  
    match e:  
        case ast.Constant(value=int(n)):  
            return str(n)          # emits nothing
```

```
compile_expr( 3 ) → returns "3"  lines: unchanged
```

3 is already a perfectly good operand, so there is nothing to compute. The “name” of its value is just the text 3.

The recursion *is* the bottom-up walk.

```
BINOPS = {ast.Add: "add", ast.Sub: "sub", ast.Mult: "mul"}

case ast.BinOp(left=l, op=op, right=r):
    a = compile_expr(l, out)      # name of the left value
    b = compile_expr(r, out)      # name of the right value
    t = out.fresh()               # a new name for my result
    out.emit(f"{t} = {BINOPS[type(op)]} i32 {a}, {b}")
    return t
```

```
compile_expr( 3 * (3 + 4) )
```

Children first, then me. That is slide 7's “bottom-up”, and nobody wrote a schedule: the order of the recursive calls *is* the order of the instructions.

Negation: the target has no **neg**.

```
case ast.UnaryOp(op=ast.USub(), operand=x):  
    a = compile_expr(x, out)  
    t = out.fresh()  
    out.emit(f"{t} = sub i32 0, {a}")  
    return t
```

```
print(-(2 + 5))
```

```
%t1 = add i32 2, 5  
%t2 = sub i32 0, %t1      ; -x is 0 - x
```

LLVM has no negate instruction, so $-x$ becomes $0 - x$. A compiler translates into what the target *has*, not what the source says.

Statements and the wrapper.

```
def compile_stmt(s, out):
    match s:
        case ast.Expr(value=ast.Call(func=ast.Name(id="print"), args=[arg])):
            v = compile_expr(arg, out)
            out.emit(f"call void @print_int(i32 {v})")
```

```
compile_program("print(3 * (3 + 4))")
= slide 11's file, character for
character
```

```
declare void @print_int(i32)

define i32 @main() {
entry:
    %t1 = add i32 3, 4
    %t2 = mul i32 3, %t1
    call void @print_int(i32 %t2)
    ret i32 0
}
```

`compile_stmt` adds one instruction per `print`. `compile_program` wraps the list in slide 11's skeleton. The file you read on slide 11, nobody writes by hand again.

Live · 5 minutes

Fill the hole, then run it.

```
$ cat p1.py
print(3 * (3 + 4))
print(-(2 + 5))
$ python3 cocoa2.py p1.py > p1.ll
$ clang p1.ll runtime.c -o p1 && ./p1
21
-7
```

Tree in. Instructions out. Native binary. You did the middle.

What we just wrote.

p1.ll · entry block only · comments added on the slide

```
%t1 = add i32 3, 4           ; 3 + 4
%t2 = mul i32 3, %t1         ; 3 * (...)
call void @print_int(i32 %t2) ; print(...)
                                %t3 = add i32 2, 5           ; 2 + 5
%t4 = sub i32 0, %t3         ; -(...)
call void @print_int(i32 %t4) ; print(...)
```

One line per operator, one per `print`, and no line for any number. Every instruction came from one tree node.

What a compiler promises.

Input: a tree. (Someone else's problem where it came from.)

Output: a program that produces the *same answers* as the source.

Anything else — speed, size, prettiness — is negotiable.

Every value today was born, used once, and thrown away.

Go back to slide 21: `%t1` is read by the very next instruction and never again. The tree told us exactly when we were finished with it — which is why `fresh()` never has to reuse a name, and why we never once touched slide 8's memory strip.

On Monday, one line breaks that:

```
x: int = 3
print(x * (x + 4))
```

`x` has to survive *past* the expression that produced it, and be *replaced* later. A tree has no way to say “same name, different value, further down.” **So where does that 3 live? Bring an answer Monday.**

PA1 — out now, due Fri 10/9.



Today's language plus: variables with declared types, booleans and comparisons, `if`, `while` (Monday's lecture).

Starter: `cocoa/`, `runtime.c`, the reference interpreter, 20 public tests.

Submit: your compiler + your own test suite (graded separately).

Your PA1 spec has a *twist* — read the first page of the handout.